

Лабораторна робота № 10

Мета: Ознайомлення з порожніми патернами проектування, практичне застосування патернів та їх реалізація в обраній мові програмування.

Завдання:

ЗАВДАННЯ 1

Відповідно до **варіанту (додаток А, патерн 1)** та **предметної області (додаток В)** реалізувати приклад застосування породжуючого патерну проектування на обраній мові програмування в консольному додатку.

ЗАВДАННЯ 2 Базове розгортання проекту на віддаленому сервері

Відповідно до **варіанту (додаток А, патерн 2)** та **предметної області (додаток В)** реалізувати приклад застосування породжуючого патерну проектування на обраній мові програмування в консольному додатку.

ЗАВДАННЯ 3

Додати до попередньої реалізації WebApi з ЛР4-6 застосування породжуючого патерну проектування Factory Method

Хід роботи:

Завдання 1

Реалізовано паттерн одинак до предметної області системи керування підписками:

Створено клас SubLogger, який побудований за принципом цього паттерну:

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
namespace Task1  
{  
    public class SubLogger
```

					ЛР.ОК24.ПІ231.03.10		
Змін.	Аркуш	№ докум.	Підпис	Дата	Creational Design Patterns		
Розробив	Башук О. Ю.						
Перевірів	Жереб Д. В.						
Н.контр.							
Затвер.							
					Літ	Аркуш	Аркушів
					н	1	16
					Х П К		

```

{
    private static SubLogger _instance;
    private string path;
    public static SubLogger Instance
    {
        get => _instance ?? (_instance = new SubLogger());
    }
    private SubLogger()
    {
        path = "SubscriptionLog.log";
        Log("Створено об'єк логгера");
    }
    public void Log(string message)
    {
        string str = $"{DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss")} -
LOG: {message}" + Environment.NewLine;
        File.AppendAllText(path, str);
    }
}

```

Створено також ще один клас, який побудований за цим принципом:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace Task1
{
    public class SubService
    {
        private static SubService _instance;
        private List<Subscription> subscriptions;
        private SubService()
        {

```

					ЛР.ОК24.ПІ231.03.10	Арк.
						2
Вим.	Арк.	№ докум.	Підпис	Дата		

```

        subscriptions = new List<Subscription>();
        SubLogger.Instance.Log("Створено сервіс");
    }

    public static SubService Instance
    {
        get => _instance ?? (_instance = new SubService());
    }

    public void AddSub(Subscription subscription)
    {
        subscriptions.Add(subscription);
        SubLogger.Instance.Log("Додано підписку: " + subscription);
    }

    public void RemoveSub(string id)
    {
        var sub = subscriptions.FirstOrDefault(s => s.Id == id);
        if (sub is null)
        {
            SubLogger.Instance.Log($"Помилка отримання підписки за id: {id}");
            throw new ArgumentException($"Не знайдено за id: {id}");
        }
        subscriptions.Remove(sub);
        SubLogger.Instance.Log($"Видалено підписку: {sub}");
    }

    public void Print()
    {
        if (subscriptions.Count == 0)
        {
            SubLogger.Instance.Log("Споба переглянути порожній список");
            throw new Exception("Список пустий");
        }
        foreach (var subscription in subscriptions)
        {
            Console.WriteLine(subscription);
        }
    }

```

					ЛР.ОК24.ПІ231.03.10	Арк.
						3
Вим.	Арк.	№ докум.	Підпис	Дата		

```

        SubLogger.Instance.Log("Переглянуто вміст списку");
    }
}

```

Для організації роботи використовувалась модель підписки:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace Task1
{
    public class Subscription
    {
        public string Id { get; private set; }
        public string Service { get; set; }
        public double Cost { get; set; }
        public Subscription(string service = "", double cost = 0)
        {
            Id = Guid.NewGuid().ToString().Substring(0,5);
            Service = service;
            Cost = cost;
        }
        public void Init()
        {
            do
            {
                Console.WriteLine("Введіть назву сервісу:");
                Service = Console.ReadLine();
            } while (string.IsNullOrEmpty(Service));
            Console.WriteLine("Введіть вартість");
            bool valid = false;
            double cost = 0;
            while (!valid)

```

					ЛР.ОК24.ПІ231.03.10	Арк.
						4
Вим.	Арк.	№ докум.	Підпис	Дата		

```

        valid = double.TryParse(Console.ReadLine(), out cost) && cost > 0;
        Cost = cost;
    }
    public override string ToString() =>
        $"Підписка #{Id} - {Service}:{Cost}";
    }
}

```

Організовано перебіг програми в Program.cs:

```

using System.Text;
using System.Text.Unicode;
using Task1;
class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = Encoding.UTF8;
        int num;
        SubLogger.Instance.Log("Початок запуску програми");
        do
        {
            num = -1;
            Console.WriteLine("Введіть дію, яку бажаєте здійснити" +
                "\n1 - Додати підписку" +
                "\n2 - Видалити підписку" +
                "\n3 - Переглянути всі підписки" +
                "\nІнше - вихід");

            int.TryParse(Console.ReadLine(), out num);
            switch (num)
            {
                case 1:
                {
                    Subscription subscription = new Subscription();
                    subscription.Init();

```

					ЛР.ОК24.ПІ231.03.10	Арк.
						5
Вим.	Арк.	№ докум.	Підпис	Дата		

```

        SubService.Instance.AddSub(subscription);
    }break;
case 2:
    {
        try
        {
            Console.WriteLine("Введіть id: ");
            string id = Console.ReadLine();
            SubService.Instance.RemoveSub(id);
        }
        catch (ArgumentException aex)
        {
            Console.WriteLine(aex.Message);
        }
    }break;
case 3:
    {
        try
        {
            Console.WriteLine("Вміст:");
            SubService.Instance.Print();
        }
        catch(Exception ex)
        {
            Console.WriteLine(ex.Message);
        }
    }break;
    }
} while (num >= 1 && num <= 3);
SubLogger.Instance.Log("Кінець роботи програми");
}
}

```

Протестовано роботу програми:

					ЛР.ОК24.ПІ231.03.10	Арк.
						6
Вим.	Арк.	№ докум.	Підпис	Дата		

```

Введіть дію, яку бажаєте здійснити
1 - Додати підписку
2 - Видалити підписку
3 - Переглянути всі підписки
Інше - вихід
1
Введіть назву сервісу:
Netflix
Введіть вартість
1000
Введіть дію, яку бажаєте здійснити
1 - Додати підписку
2 - Видалити підписку
3 - Переглянути всі підписки
Інше - вихід
1
Введіть назву сервісу:
Megogo
Введіть вартість
500
Введіть дію, яку бажаєте здійснити
1 - Додати підписку
2 - Видалити підписку
3 - Переглянути всі підписки
Інше - вихід
1
Введіть назву сервісу:
Google
Введіть вартість
1000
Введіть дію, яку бажаєте здійснити
1 - Додати підписку
2 - Видалити підписку
3 - Переглянути всі підписки
Інше - вихід
1
Введіть назву сервісу:
PlayStation
Введіть вартість
1000

```

Рисунок 1 – Створення чотирьох підписок

```

Введіть дію, яку бажаєте здійснити
1 - Додати підписку
2 - Видалити підписку
3 - Переглянути всі підписки
Інше - вихід
3
Вміст:
Підписка #3574b - Netflix:1000
Підписка #ac77b - Megogo:500
Підписка #052d4 - Google:1000
Підписка #5e03f - PlayStation:1000

```

Рисунок 2 – Переглянуто наявні підписки

```

Введіть дію, яку бажаєте здійснити
1 - Додати підписку
2 - Видалити підписку
3 - Переглянути всі підписки
Інше - вихід
2
Введіть id:
3574b

```

Рисунок 3 – Видалено підписку

```

Введіть дію, яку бажаєте здійснити
1 - Додати підписку
2 - Видалити підписку
3 - Переглянути всі підписки
Інше - вихід
3
Вміст:
Підписка #ac77b - Megogo:500
Підписка #052d4 - Google:1000
Підписка #5e03f - PlayStation:1000
Введіть дію, яку бажаєте здійснити
1 - Додати підписку
2 - Видалити підписку
3 - Переглянути всі підписки
Інше - вихід

```

Рисунок 4 – Повторно переглянуто підписки

```

2025-11-20 02:27:58 - LOG: Створено об'єк логгера
2025-11-20 02:27:58 - LOG: Початок запуску програми
2025-11-20 02:28:10 - LOG: Створено сервіс
2025-11-20 02:28:10 - LOG: Додано підписку: Підписка #3574b - Netflix:1000
2025-11-20 02:28:24 - LOG: Додано підписку: Підписка #ac77b - Megogo:500
2025-11-20 02:28:34 - LOG: Додано підписку: Підписка #052d4 - Google:1000
2025-11-20 02:28:56 - LOG: Додано підписку: Підписка #5e03f - PlayStation:1000
2025-11-20 02:29:12 - LOG: Переглянуто вміст списку
2025-11-20 02:29:24 - LOG: Видалено підписку: Підписка #3574b - Netflix:1000
2025-11-20 02:29:25 - LOG: Переглянуто вміст списку
2025-11-20 02:32:44 - LOG: Кінець роботи програми

```

					ЛР.ОК24.ПІ231.03.10	Арк.
						7
Вим.	Арк.	№ докум.	Підпис	Дата		

Завдання 2

Реалізовано використання паттерну прототип до попередньої програми:

Додано інтерфейс для подальшої реалізації класом:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace Task1.Interfaces
{
    public interface IPrototype
    {
        IPrototype Clone();
    }
}
```

Визначено класом метод Clone():

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task1.Interfaces;
namespace Task1.Model
{
    public class Subscription : IPrototype
    {
        public string Id { get; private set; }
        public string Service { get; set; }
        public double Cost { get; set; }
        public Subscription(string service = "", double cost = 0)
        {
            Id = Guid.NewGuid().ToString().Substring(0,5);
            Service = service;
            Cost = cost;
        }
        public void Init()
        {
            do
            {
                Console.WriteLine("Введіть назву сервісу:");
                Service = Console.ReadLine();
            } while (string.IsNullOrEmpty(Service));
            Console.WriteLine("Введіть вартість");
            bool valid = false;
            double cost = 0;
            while (!valid)
            {
                valid = double.TryParse(Console.ReadLine(), out cost) && cost > 0;
                Cost = cost;
            }
        }
    }
}
```

					ЛР.ОК24.ПІ231.03.10	Арк.
						8
Вим.	Арк.	№ докум.	Підпис	Дата		

```

    }
    public override string ToString() =>
        $"Підписка #{Id} - {Service}:{Cost}";
    public IPrototype Clone()
    {
        var clone = (Subscription)this.MemberwiseClone();
        clone.Id = Guid.NewGuid().ToString().Substring(0,5);
        return clone;
    }
}
}
}

```

Додано метод в сервіс:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task1.Model;
namespace Task1.Services
{
    public class SubService
    {
        private static SubService _instance;
        private List<Subscription> subscriptions;
        private SubService()
        {
            subscriptions = new List<Subscription>();
            SubLogger.Instance.Log("Створено сервіс");
        }
        public static SubService Instance
        {
            get => _instance ?? (_instance = new SubService());
        }
        public void AddSub(Subscription subscription)
        {
            subscriptions.Add(subscription);
            SubLogger.Instance.Log("Додано підписку: " + subscription);
        }
        public void RemoveSub(string id)
        {
            var sub = subscriptions.FirstOrDefault(s => s.Id == id);
            if (sub is null)
            {
                SubLogger.Instance.Log($"Помилка отримання підписки за id: {id}");
                throw new ArgumentException($"Не знайдено за id: {id}");
            }
            subscriptions.Remove(sub);
            SubLogger.Instance.Log($"Видалено підписку: {sub}");
        }
        public void Print()
        {

```

					ЛР.ОК24.ПІ231.03.10	Арк. 9
Вим.	Арк.	№ докум.	Підпис	Дата		

```

        if (subscriptions.Count == 0)
        {
            SubLogger.Instance.Log("Споба переглянути порожній список");
            throw new Exception("Список пустий");
        }
        foreach (var subscription in subscriptions)
        {
            Console.WriteLine(subscription);
        }
        SubLogger.Instance.Log("Переглянуто вміст списку");
    }
    public void CreateFromExist(string id)
    {
        var sub = subscriptions.FirstOrDefault(s => s.Id == id);
        if(sub is null)
        {
            SubLogger.Instance.Log("Не знайдено підписку при спробі копіювання");
            throw new ArgumentException($"Не знайдено підписку із вказаним id: {id}");
        }
        subscriptions.Add((Subscription)sub.Clone());
    }
}

```

Змінено вміст основної програми:

```

using System.Text;
using System.Text.Unicode;
using Task1;
using Task1.Model;
using Task1.Services;
class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = Encoding.UTF8;
        int num;
        SubLogger.Instance.Log("Початок запуску програми");
        do
        {
            num = -1;
            Console.WriteLine("Введіть дію, яку бажаєте здійснити" +
                "\n1 - Додати підписку" +
                "\n2 - Видалити підписку" +
                "\n3 - Переглянути всі підписки" +
                "\n4 - Створити на основі існуючої" +
                "\nІнше - вихід");
            int.TryParse(Console.ReadLine(), out num);
            switch (num)
            {
                case 1:
                {
                    Subscription subscription = new Subscription();

```

					ЛР.ОК24.ПІ231.03.10	Арк.
						10
Вим.	Арк.	№ докум.	Підпис	Дата		

```

        subscription.Init();
        SubService.Instance.AddSub(subscription);
    }break;
case 2:
    {
        try
        {
            Console.WriteLine("Введіть id: ");
            string id = Console.ReadLine();
            SubService.Instance.RemoveSub(id);
        }
        catch (ArgumentException aex)
        {
            Console.WriteLine(aex.Message);
        }
    }break;
case 3:
    {
        try
        {
            Console.WriteLine("Вміст:");
            SubService.Instance.Print();
        }
        catch(Exception ex)
        {
            Console.WriteLine(ex.Message);
        }
    }break;
case 4:
    {
        try
        {
            Console.WriteLine("Введіть id:");
            string id = Console.ReadLine();
            SubService.Instance.CreateFromExist(id);
        }
        catch(ArgumentException aex)
        {
            Console.WriteLine(aex.Message);
        }
    }break;
    }
} while (num >= 1 && num <= 4);

SubLogger.Instance.Log("Кінець роботи програми");
}
}

```

Протестовано роботу програми:

					ЛР.ОК24.ПІ231.03.10	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		11

```

Введіть дію, яку бажаєте здійснити
1 - Додати підписку
2 - Видалити підписку
3 - Переглянути всі підписки
4 - Створити на основі існуючої
Інше - вихід
1
Введіть назву сервісу:
Netflix
Введіть вартість
1000
Введіть дію, яку бажаєте здійснити
1 - Додати підписку
2 - Видалити підписку
3 - Переглянути всі підписки
4 - Створити на основі існуючої
Інше - вихід
1
Введіть назву сервісу:
Megogo
Введіть вартість
500
Введіть дію, яку бажаєте здійснити
1 - Додати підписку
2 - Видалити підписку
3 - Переглянути всі підписки
4 - Створити на основі існуючої
Інше - вихід
1
Введіть назву сервісу:
Google
Введіть вартість
1000
Введіть дію, яку бажаєте здійснити
1 - Додати підписку
2 - Видалити підписку
3 - Переглянути всі підписки
4 - Створити на основі існуючої
Інше - вихід
1
Введіть назву сервісу:
PlayStation
Введіть вартість
1000

```

Рисунок 6 – Створення чотрьох підписок

```

Введіть дію, яку бажаєте здійснити
1 - Додати підписку
2 - Видалити підписку
3 - Переглянути всі підписки
4 - Створити на основі існуючої
Інше - вихід
3
Вміст:
Підписка #b4b77 - Netflix:1000
Підписка #f1cae - Megogo:500
Підписка #8e944 - Google:1000
Підписка #38a96 - PlayStation:1000

```

Рисунок 7 – переглянуто список створених підписок

```

Введіть дію, яку бажаєте здійснити
1 - Додати підписку
2 - Видалити підписку
3 - Переглянути всі підписки
4 - Створити на основі існуючої
Інше - вихід
4
Введіть id:
b4b77

```

Рисунок 8 – Копіювання першої підписки

```

Введіть дію, яку бажаєте здійснити
1 - Додати підписку
2 - Видалити підписку
3 - Переглянути всі підписки
4 - Створити на основі існуючої
Інше - вихід
3
Вміст:
Підписка #b4b77 - Netflix:1000
Підписка #f1cae - Megogo:500
Підписка #8e944 - Google:1000
Підписка #38a96 - PlayStation:1000
Підписка #abfd4 - Netflix:1000

```

Рисунок 9 – Знову переглянуто вміст

Завдання 3

Створено інтерфейс для фабрик по створенню повідомлень:

```

using LW4_task_3.Models.Entities;
namespace LW4_task_3.FactoryMethod.Interface
{
    public interface IMessageFactory
    {
        MessageItem CreateMessage(SubscriptionItem sub);
    }
}

```

					ЛР.ОК24.ПІ231.03.10	Арк.
						12
Вим.	Арк.	№ докум.	Підпис	Дата		

```

}
    Створено фабрики, що реалізують даний інтерфейс:
using LW4_task_3.FactoryMethod.Interface;
using LW4_task_3.Models.Entities;
namespace LW4_task_3.FactoryMethod.ConcreteFactory
{
    public class ActiveMessageFactory : IMessageFactory
    {
        public MessageItem CreateMessage(SubscriptionItem sub) =>
            new MessageItem
            {
                Title = $"Підписка {sub.Service} активована",
                SubId = sub.Id,
                OwnerId = sub.OwnerId
            };
    }
}
using LW4_task_3.FactoryMethod.Interface;
using LW4_task_3.Models.Entities;

namespace LW4_task_3.FactoryMethod.ConcreteFactory
{
    public class ArchivedMessageFactory : IMessageFactory
    {
        public MessageItem CreateMessage(SubscriptionItem sub) =>
            new MessageItem
            {
                Title = $"Підписку {sub.Service} додано в архів",
                SubId = sub.Id,
                OwnerId = sub.OwnerId
            };
    }
}
using LW4_task_3.FactoryMethod.Interface;
using LW4_task_3.Models.Entities;

namespace LW4_task_3.FactoryMethod.ConcreteFactory
{
    public class ExpectationMessageFactory : IMessageFactory
    {
        public MessageItem CreateMessage(SubscriptionItem sub) =>
            new MessageItem
            {
                Title = $"Підписка {sub.Service} створена, та ще не активована",
                SubId = sub.Id,
                OwnerId = sub.OwnerId
            };
    }
}
using LW4_task_3.FactoryMethod.Interface;
using LW4_task_3.Models.Entities;

```

					ЛР.ОК24.ПІ231.03.10	Арк.
						13
Вим.	Арк.	№ докум.	Підпис	Дата		

```

namespace LW4_task_3.FactoryMethod.ConcreteFactory
{
    public class OverdueMessageFactory : IMessageFactory
    {
        public MessageItem CreateMessage(SubscriptionItem sub) =>
            new MessageItem
            {
                Title = $"Підписка {sub.Service} прострочена, порібно оплатити",
                SubId = sub.Id,
                OwnerId = sub.OwnerId
            };
    }
}

    Створено сервіс для створення фабрики по статусу підписки:
using LW4_task_3.FactoryMethod.Interface;
using LW4_task_3.Models.Entities;
using ZstdSharp.Unsafe;
using LW4_task_3.FactoryMethod.ConcreteFactory;
namespace LW4_task_3.FactoryMethod.Service
{
    public class MessageFactoryService
    {
        private readonly IServiceProvider _provider;

        public MessageFactoryService(IServiceProvider provider)
        {
            _provider = provider;
        }

        public IMessageFactory GetFactory(SubStatus subStatus)
        {
            return subStatus switch
            {
                SubStatus.Expectation => _provider.GetRequiredService<ExpectationMessageFactory>(),
                SubStatus.Active => _provider.GetRequiredService<ActiveMessageFactory>(),
                SubStatus.Overdue => _provider.GetRequiredService<OverdueMessageFactory>(),
                SubStatus.Archived => _provider.GetRequiredService<ArchivedMessageFactory>()
            };
        }
    }
}

    Зареєстровано фабрики та сервіс в Program.cs:
builder.Services.AddSingleton<ExpectationMessageFactory>();
builder.Services.AddSingleton<ActiveMessageFactory>();
builder.Services.AddSingleton<OverdueMessageFactory>();
builder.Services.AddSingleton<ArchivedMessageFactory>();
builder.Services.AddSingleton<MessageFactoryService>();

    Додано до методів CreateAsync, UpdateAsync MessageService частину отримання
фабрики та створено повідомлення:

```

					ЛР.ОК24.ПІ231.03.10	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		14

var factory = _messageFactoryService.GetFactory(element.Status);
await _messageRepository.CreateAsync(factory.CreateMessage(element));
Протестовано функціонал після використання паттерну фабричний метод:



Рисунок 10 – Створення нової підписки

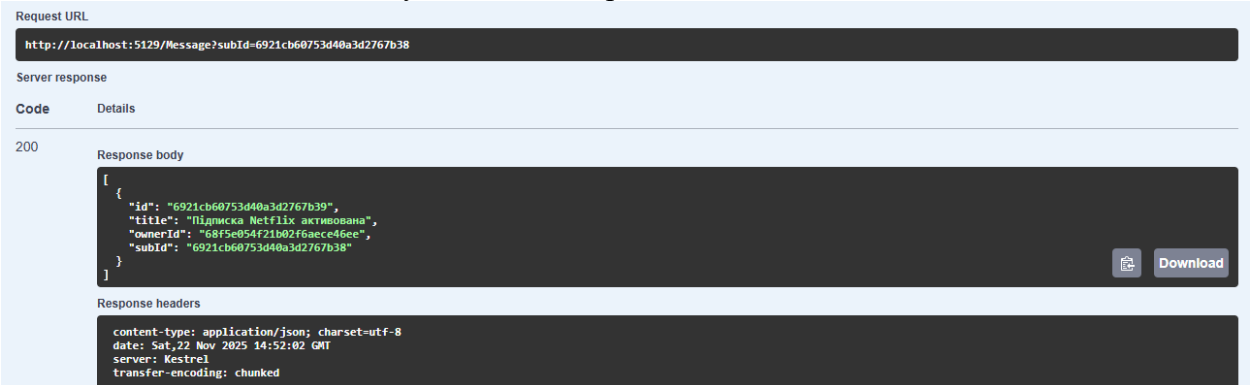


Рисунок 11 – Отримано повідомлення за Id



Рисунок 12 – Оновлено підписку

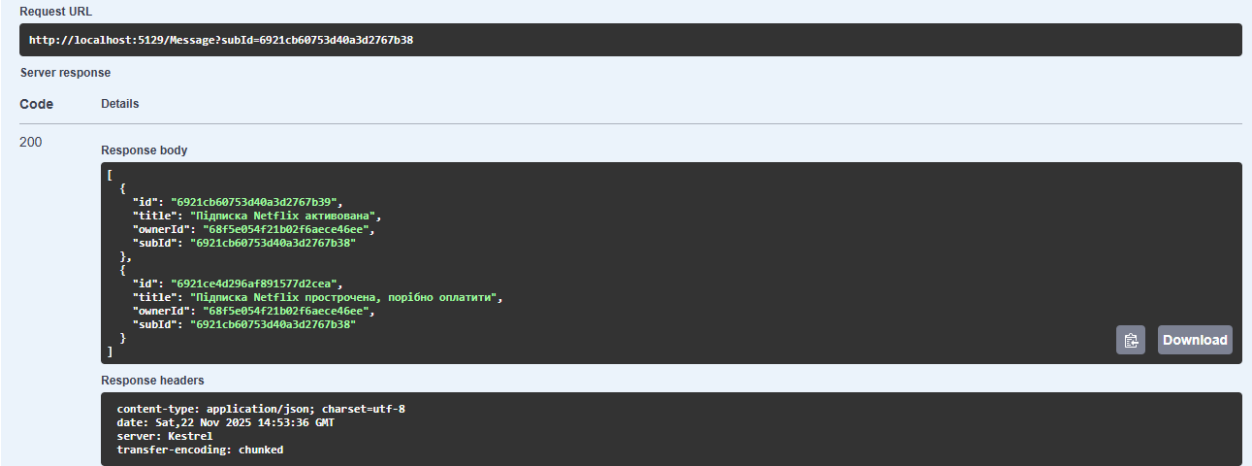


Рисунок 13 – Перевірено повідомлення про підписки

ВИСНОВОК:

					ЛР.ОК24.ПІ231.03.10	Арк.
						15
Вим.	Арк.	№ докум.	Підпис	Дата		

На даній лабораторній роботі я вивчив основні породжувальні паттерни та як із ними працювати, набув практичних навичків по використанню паттернів один, прототип та фабричний метод

					ЛР.ОК24.ПІ231.03.10	Арк.
						16
Вим.	Арк.	№ докум.	Підпис	Дата		